Optimising Attention on a CPU

CS6886: Systems for Deep Learning, IIT Madras. Due: **[TBD]**.

Task

You are given a naive C implementation of single-head attention, $O = \text{softmax}(QK^\top/\sqrt{d})\,V$, where Q, K, V and O are $N \times d$ row-major float matrices and the softmax is applied to each row. Make it fast on your own machine, step by step, and explain every result with measurements.

What you are given

- `src/attn_naive.c`: the naive kernel and a small driver that reads an input file, times the kernel (median of 5 runs after a warm-up) and writes the output.
- `src/attn_<variant>.c`: a placeholder file for each variant, with the same driver. Write your code in its `attention()` function.
- `input/`: four inputs, `qkv_N<N>_d<d>.bin`, for (N, d) = (1024, 64), (2048, 64), (4096, 64) and (4096, 128).
- `Makefile`: `make <variant>` (e.g. `make naive`, `make tiled`) compiles one variant and reports its wall-clock time on every input; `make all` does this for every variant; `make check` checks that every variant's output matches naive's within a relative error of 10^{-4}. Set `CC` at the top of the Makefile to the path of the **latest clang**.
- Templates: `machine_spec.txt`, `results.csv`, and the `plots/` folder.

The variants are `naive`, `loops`, `tiled`, `fused`, `best` and `omp`. Count $4N^2d$ floating-point operations per call when computing GFLOP/s. Write all the code yourself: no BLAS or other math libraries.

Questions

Q1. Your machine

Fill in `machine_spec.txt`: CPU model, cores (physical, logical, performance and efficiency), clock frequencies, SIMD width and FMA units, cache sizes, memory type and size. From these, compute the peak FP32 GFLOP/s (one core and all cores), the memory bandwidth and the ridge point. Show your calculations in the report.

Q2. Naive baseline

Run `make naive`. For each input, report the time, GFLOP/s and percentage of peak. How does the time change with N and with d? Explain where the time goes.

Q3. Optimisations

Write each variant, run it, and report the same measurements as Q2 plus the speedup over the previous variant. `make check` must pass. Explain what each change does and what limits performance afterwards.

(a) `attn_loops.c`: reorder the loops so the compiler can vectorise them.

(b) `attn_tiled.c`: cache and register tiling.

(c) `attn_fused.c`: fuse the three steps row by row so the $N \times N$ matrix is never stored.

(d) `attn_best.c`: anything you want. Get the maximum single-threaded speedup you can, and explain what you did and why it works.

Q4. OpenMP

Speed up your best variant with OpenMP in `attn_omp.c`. Run it with different thread counts (e.g. `OMP_NUM_THREADS=4 make omp`) and report the time, speedup and parallel efficiency. Explain the scaling you see.

Q5. Roofline

Plot the roofline of your machine using your Q1 numbers, and place every variant on it at $N = 4096$, $d = 64$. Explain how you computed each variant's arithmetic intensity and what limits each variant.

Submission

Submit a link to a **public GitHub repository** with exactly this structure:

Path	Contents
`src/`	`attn_naive.c` and all your variants
`input/`	The given inputs
`machine_spec.txt`	Your filled-in machine specification
`results.csv`	Your filled-in results (add rows for the thread counts you tried)
`plots/roofline.png`	The roofline with all variants
`plots/time_N<N>_d<d>.png`	One plot per input: time of every variant
`Makefile`	The given Makefile
`assignment_handout.pdf`	This handout
`report.pdf`	Your answers to Q1–Q5, in any format